arXiv:2310.02226v3 [cs.CL] 21 Apr 2024

THINK BEFORE YOU SPEAK: TRAINING LANGUAGE MODELS WITH PAUSE TOKENS

Sachin Goyal[*†]
Machine Learning Department
Carnegie Mellon University
sachingo@andrew.cmu.edu

Ziwei Ji
Google Research, NY
ziweiji@google.com

Ankit Singh Rawat
Google Research, NY
ankitsrawat@google.com

Aditya Krishna Menon
Google Research, NY
adityakmenon@google.com

Sanjiv Kumar
Google Research, NY
sanjivk@google.com

Vaishnavh Nagarajan[†]
Google Research, NY
vaishnavh@google.com

ABSTRACT

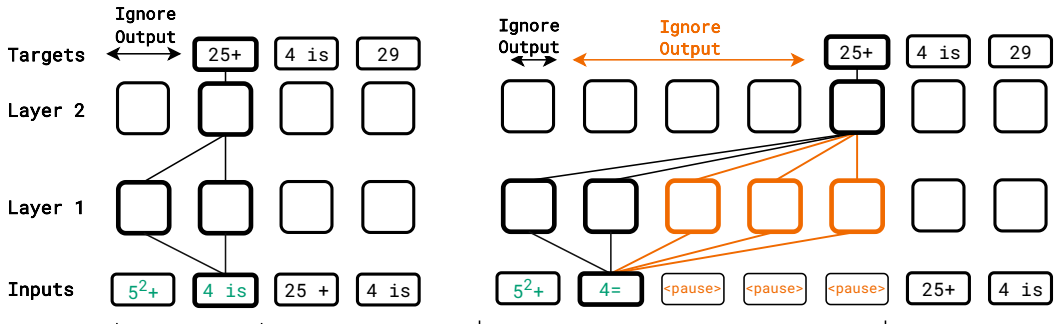
Transformer-based language models generate responses by producing a series of tokens in immediate succession: the $(K+1)^{\text{th}}$ token is an outcome of manipulating K hidden vectors per layer, one vector per preceding token. What if instead we were to let the model manipulate say, $K+10$ hidden vectors, before it outputs the $(K+1)^{\text{th}}$ token? We operationalize this idea by performing training and inference on language models with a (learnable) *pause* token, a sequence of which is appended to the input prefix. We then delay extracting the model's outputs until the last pause token is seen, thereby allowing the model to process extra computation before committing to an answer. We empirically evaluate *pause-training* on decoder-only models of 1B and 130M parameters with causal pretraining on C4, and on downstream tasks covering reasoning, question-answering, general understanding and fact recall. Our main finding is that inference-time delays show gains on our tasks when the model is both pretrained and finetuned with delays. For the 1B model, we witness gains on eight tasks, most prominently, a gain of 18% EM score on the QA task of SQuAD, 8% on CommonSenseQA and 1% accuracy on the reasoning task of GSM8k. Our work raises a range of conceptual and practical future research questions on making delayed next-token prediction a widely applicable new paradigm.

1 INTRODUCTION

Transformer-based causal language models generate tokens one after the other in immediate succession. To generate the $(K+1)^{\text{th}}$ token, the model consumes the K previous tokens, and proceeds layer by layer, computing K intermediate vectors in each hidden layer. Each vector in itself is the output of a module (consisting of self-attention and multi-layer-perceptrons) operating on the previous layer's output vectors. However sophisticated this end-to-end process may be, it abides by a peculiar constraint: the number of operations determining the next token is limited by the number of tokens seen so far. Arguably, this was the most natural design choice when the Transformer was first conceived by Vaswani et al. (2017). But in hindsight, one may wonder whether for some inputs, the $(K+1)^{\text{th}}$ token demands $K+M$ Transformer operations in each layer (for $M>0$), which cannot be met by the arbitrarily constrained K operations per layer. This paper explores one way to free the Transformer of this arbitrary per-layer computational constraint.

The approach we study is to append dummy tokens into a decoder-only model's input, thereby *delaying* the model's output. Specifically, we select a (learnable) pause token (denoted `<pause>`) and append one or more copies of `<pause>` as a sequence to the input. We simply ignore the model's corresponding outputs until the last `<pause>` token is seen, after which we begin extracting its response.

[*]Work done in part as a Student Researcher at Google.
[†]Corresponding Authors



(a) Standard inference and finetuning

(b) Pause-inference and finetuning

Figure 1: **Standard vs. pause-inference (and finetuning)**. We consider a downstream task where, given a prefix, the decoder-only model (bidirectionally) attends to all of the prefix to generate its target answer. The rounded squares denote one Transformer operation (a self-attention and MLP) in a 2-layer Transformer. Any `Ignore Output` denotes that during inference, the corresponding output token is not extracted and thus, not fed back autoregressively; during finetuning, this output is not backpropagated through. The connecting lines denote some (not all) of the "computational pathways" within the model. Specifically, we visualize only those pathways that begin at a specific token in the prefix (here arbitrarily chosen to be "`4 is`") and end at an output token (here arbitrarily chosen to be "`25+`"). All differences between the two settings are highlighted in color. (a) In standard inference (finetuning), the model's output is extracted immediately upon seeing the last prefix token. (b) In pause-inference (and pause-finetuning), this is initiated only after appending a manually specified number of `<pause>` tokens. This introduces new computational pathways (the colored lines) between the prefix token and the output token of interest.

Crucially, we consider injecting such delays not just at inference, but also during downstream finetuning (see Fig 1) and pretraining (see Fig 2, which provides additional technical details).

A-priori, it is unclear what this simple change would bring about in practice. Optimistically, the Transformer may take advantage of a "wider" computational pathway induced by the delay. A more mundane outcome though would be that the model simply skips any delays introduced by the `<pause>` tokens. After all, neither do the `<pause>` tokens provide any additional information during inference, *nor are there sufficiently many new parameters* (barring the few embedding parameters of the single `<pause>` token) that can encode any additional information from training data. Worse still, these uninformative tokens may drown out informative signals, and hurt the model.

Partial answers to this question can be found in the literature, motivated somewhat differently. To understand where the benefits of chain-of-thought (Wei et al., 2022) come from, Lanham et al. (2023) append dummy thoughts in the form of periods ('...'), but only during inference. This, they report, does not help. Presumably, an off-the-shelf model may not have *learned* to utilize the new computational pathways offered by the inference-time delay. Burtsev et al. (2020) learn with *prepended* dummy tokens, with the orthogonal motivation of adding memory (rather than extending computation). They train with these tokens only on the target task, and observe minimal performance gains.

What then can we hope for when injecting (appended) delays on all stages of training and inference? Our work empirically evaluates this, and other key questions that come up when training the Transformer with delays. For this, we study pause-training on a 1B and 130M parameter decoder-only model, trained on C4 (Raffel et al., 2020) and finetuned on nine downstream tasks spanning extractive question answering, reasoning, general understanding and fact recall. In summary, we make the following contributions:

(1) We pose the question of *what happens if we delay a model's answer generation, and how can we execute these delays?* We design one way: training with dummy `<pause>` tokens. Accordingly, we design a pause-injected pretraining, downstream finetuning, and inference procedure.

(2) We find that on a variety of downstream tasks, training models with `<pause>` tokens during both pretraining and downstream finetuning, exhibits clear gains compared to standard end-to-end training and inference. Most notably, for the 1B model, in the SQuAD extractive question-

answering task, this approach improves the exact match score by 18%. Similarly we observe 8% gains on the general understanding task of CommonSense QA and 1% accuracy gain on the reasoning task of GSM8k over the standard model's accuracy of 7.5%.

(3) On the flip side, when we introduce <pause> tokens only in the downstream finetuning stage (on the standard pretrained model), we find that the gains show up in far fewer instances, and are relatively mild. In some instances, we even find a clear drop in performance.

(4) We also conduct a series of key ablations: (a) We find that *appending* <pause> tokens is largely better than *prepending* them, (b) perhaps unsurprisingly, for any downstream task, there is a corresponding optimal number of <pause> tokens, and (c) when decreasing the number of inference-time <pause> tokens, we find a graceful degradation of performance even though pause-training does not explicitly train for such robustness.

Overall, our work explores the new paradigm of delayed next-token generation in Transformer models, and finds that there *are* benefits to this simple change, provided the change is implemented both during pretraining and finetuning. Our preliminary step here inspires a variety of conceptual and practical future research questions, ranging from understanding how Transformer delays work mechanistically, to making pause-training more generally applicable for practice.

2 PRELIMINARIES

We briefly outline the next-token prediction process in a standard causal decoder-only language model (details in §A). Consider a vocabulary $\mathcal{V}$ and an input $\mathbf{p}_{1:K} \in \mathcal{V}^K$ of K tokens. Let f denote a Transformer-based language model, from which we sample the next token as $p_{K+1} \sim f(\mathbf{p}_{1:K})$. To achieve this, internally, each layer $l \in [1, L]$ of the Transformer produces an intermediate vector $\mathbf{v}_k^{(l)}$ corresponding to each input token. The next token, i.e., p_{K+1} is then sampled from a distribution inferred from the last vector in the last layer, $\mathbf{v}_K^{(L)}$.

On a high level, each layer in the above process can be represented as a function $T : \mathbb{R}^{D\times K} \to \mathbb{R}^{D\times K}$. Its input is a matrix of K vectors, $V_{1:K} = [\mathbf{v}_1, \ldots, \mathbf{v}_K]$, and likewise, the output, $V'_{1:K}$. This mapping itself involves two key (parameterized) modules. The first is the attention module Φ_{Attn} which takes as inputs two matrices $V_{\text{key}}, V_{\text{value}} \in \mathbb{R}^{D\times N}$ (for any N) and a "query" vector $\mathbf{v} \in \mathbb{R}^D$ to produce an output vector in $\mathbb{R}^D$. This is followed by a feedforward module $f_{\text{FF}} : \mathbb{R}^D \to \mathbb{R}^D$. Then, given the inputs $\mathbf{v}_k$, and given the layer-norm module $\Phi_{\text{LN}} : \mathbb{R}^D \to \mathbb{R}^D$, the outputs $\mathbf{v}'_k$ for $k \leq K$ can be expressed as:

$$\mathbf{a}_k = \Phi_{\text{LN}}\left(\Phi_{\text{Attn}}(V_{1:k}, V_{1:k}, \mathbf{v}_k) + \mathbf{v}_k\right) \tag{1}$$

$$\mathbf{v}'_k = \Phi_{\text{LN}}\left(\Phi_{\text{FF}}(\mathbf{a}_k) + \mathbf{a}_k\right). \tag{2}$$

Observe here that the kth output $\mathbf{v}'_k$ is obtained by manipulating exactly the k previous hidden embeddings in the same layer, $V_{1:k}$.

3 PAUSE-TRAINING

In the current paradigm of language models, we compute exactly K embeddings $\mathbf{v}_1, \ldots \mathbf{v}_K$ in each layer, before generating the $(K+1)^{\text{th}}$ token, p_{K+1}. Our premise is that this limit of K operations is an arbitrary one. Instead, we wish to expend more than K operations towards producing the next token, p_{K+1}. While something to this effect could be achieved by increasing the number of attention heads in each layer, we are interested in an orthogonal approach that introduces hardly any parameters into the network. The idea is to synthetically increase the input sequence length by appending M dummy tokens to the input, thus delaying the model's next response by M tokens of input. In effect, this M-token-delay lets the model manipulate an additional set of M intermediate vectors $\mathbf{v}_{K+1}, \ldots, \mathbf{v}_{K+M}$ before committing to its next (output) token, p_{K+1}. Intuitively, these vectors could provide a richer representation of the input (e.g., by attending differently), thus resulting in a better next token from the model. We visualize this in Figure 1.

3.1 LEARNING AND INFERENCE WITH THE <pause> TOKEN

A simple choice for the dummy tokens are special characters such as '.' or '#', as Lanham et al. (2023) chose for inference. But to prevent the model from confounding the role of delays with the